API Reference

All endpoints return JSON. Successful responses include `"success": true`. Application-level failures return `"success": false` with an `"exception"` message; some malformed requests also use HTTP `400`.

The explorer UI now uses `POST` + JSON for mutating actions and eval/transaction calls. A few legacy `GET` forms are still accepted for compatibility, but new integrations should use the documented `POST` routes.

Conventions

- OOP-backed object responses use the `object_view` shape from `gemstone_p/object_view.py`.
- Many browser/helper windows carry dictionary/class/method context explicitly so follow-up actions do not need to guess class locations.
- The SPA may send an internal `X-GS-Channel` header so separate windows can use isolated GemStone session families. API consumers normally do not need to set it.

UI And Root Metadata

`GET /`

Serves the single-page explorer UI.

`GET /ids`

Returns well-known startup OOPs.

```
{
  "persistentRootId": 123,
  "globalsId": 456,
  "gemStoneSystemId": 789
}
```

If startup OOP resolution fails, this route returns HTTP `500` and includes a `preflight` object matching `GET /connection/preflight`, so the SPA can open the Connection window with the attempted target and suggested fixes.

`GET /version`

Returns the explorer app version together with the current Stone and Gem version strings.

```
{
  "success": true,
  "app": "1.0.0",
  "stone": "3.7.5",
```

```
  "gem": "3.7.5"
}
```

GET /healthz

Machine-friendly health probe for the running explorer process and GemStone connection.

Healthy response:

```
{
  "success": true,
  "status": "ok",
  "app": "1.0.0",
  "stone": "3.7.5",
  "gem": "3.7.5"
}
```

If the GemStone session cannot be opened, this route returns HTTP `503` with `"status": "error"` and an `"exception"` message.

GET /connection/preflight

Returns connection-oriented diagnostics for the current process configuration. This combines the effective GemStone target from `session.py`, a lightweight runtime version check, and a best-effort local `gslist -lcv` probe plus suggestions such as `export GS_STONE=seaside`.

Healthy response shape:

```
{
  "success": true,
  "status": "ok",
  "app": "1.0.0",
  "stone": "3.7.5",
  "gem": "3.7.5",
  "connection": {
    "configured": {
      "stone": "seaside",
      "host": "localhost",
      "mode": "local-stone-name",
      "effectiveTarget": "seaside"
    },
    "probe": {
      "availableStones": ["seaside"]
    },
    "suggestions": []
  }
}
```

When the runtime session cannot be opened, this route still returns `200` with `"status": "error"` so the UI can surface the attempted target and suggested fixes.

`GET /diagnostics`

Extended runtime diagnostics for support/debugging. This includes version data, Python/platform metadata, a safe session-broker snapshot, and connection-preflight context. The About window also uses this response as the server half of its downloadable support bundle.


```
{
  "success": true,
  "status": "ok",
  "app": "1.0.0",
  "stone": "3.7.5",
  "gem": "3.7.5",
  "runtime": {
    "python": "3.12.2",
    "implementation": "CPython",
    "platform": "Darwin-24.0.0-arm64"
  },
  "sessionBroker": {
    "defaultAutoBegin": null,
    "managedSessionCount": 3,
    "channels": [
      {
        "name": "object:win-1-r",
        "hasSession": true,
        "loggedIn": true
      }
    ]
  },
  "connection": {
    "configured": {
      "stone": "seaside",
      "host": "localhost",
      "mode": "local-stone-name",
      "effectiveTarget": "seaside"
    },
    "probe": {
      "availableStones": ["seaside"]
    },
    "suggestions": []
  },
  "statusHistory": [
```

```
    {
      "timestamp": "2026-04-25T18:00:00Z",
      "ok": true,
      "message": "connected",
      "count": 1
    }
  ]
}
```

Object Browser And Workspace

`GET /object/index/<oop>`

Loads an object inspector view.

Query parameters:

- `depth` — nested object depth, default `2`
- `range_instVars_from`, `range_instVars_to`
- custom-tab ranges such as `range_attributes_from`, `range_attributes_to`

Response highlights:

- `availableTabs` / `defaultTab` — backend-driven inspector tab metadata
- `customTabs` — adapter-driven custom tabs such as MagLev-style `Attributes`
- `classBrowserTarget` — exact Class Browser handoff metadata when available

`GET|POST /object/evaluate/<oop>`

Evaluates Smalltalk in the context of the target object.

Preferred `POST` body:

```
{
  "code": "self printString",
  "language": "smalltalk",
  "depth": 2,
  "ranges": {}
}
```

Response:

```
{
  "success": true,
  "result": [
    false,
    { "...": "object_view" }
  ]
}
```

When evaluation halts or raises, the result view includes debugger metadata such as `debugThreadOop`, `exceptionText`, `sourcePreview`, and `autoOpenDebugger`.

`GET /code/selectors/<oop>`

Returns the target behavior's selectors grouped by category for the object-inspector code pane.

`GET /code/code/<oop>`

Returns source for a selector in the object-inspector code pane.

Query parameters:

- `selector` — selector name

`GET /object/constants/<oop>`

Returns class/object constants as paged rows. Constant values include real object refs when possible.

Query parameters:

- `offset`
- `limit`

`GET /object/hierarchy/<oop>`

Returns the target class hierarchy used by the object inspector.

`GET /object/included-modules/<oop>`

Returns paged included-module rows with owner/module refs.

Query parameters:

- `offset`
- `limit`

`GET /object/instances/<oop>`

Returns paged instances for a class/behavior view.

Query parameters:

- `offset`
- `limit`

`GET /object/stone-version-report`

Returns the Stone version report text used by the System inspector.

`GET /object/gem-version-report`

Returns the Gem version report text used by the System inspector.

Class Browser

Read Routes

`GET /class-browser/dictionaries`

Returns visible symbol-list dictionaries.

`GET /class-browser/class-location`

Finds every matching dictionary for a class name.

Query parameters:

- `class`

Response:

```
{
  "success": true,
  "dictionary": "",
  "matches": [
    {"className": "Object", "dictionary": "Globals"},
    {"className": "Object", "dictionary": "UserGlobals"}
  ]
}
```

`dictionary` is populated only when there is exactly one match.

`GET /class-browser/classes`

Returns classes in a dictionary.

`GET /class-browser/categories`

Returns protocols/categories for a class or metaclass.

Query parameters:

- `class`
- `dictionary`
- `meta=1` for class side

`GET /class-browser/methods`

Returns selectors for a class/category.

Query parameters:

- class
- dictionary
- protocol
- meta=1

GET /class-browser/source

Returns class definition source or method source.

Query parameters:

- class
- dictionary
- selector
- meta=1

GET /class-browser/hierarchy

Returns the superclass chain with dictionary context.

GET /class-browser/versions

Returns method versions for the selected method.

Query parameters:

- class
- dictionary
- selector
- meta=1

Response:

```
{
  "success": true,
  "versions": [
    {
      "label": "version 1",
      "source": "printString\n^ 'example'",
      "methodOop": 940
    }
  ]
}
```

`methodOop` lets the Versions helper window inspect the selected historical compiled method directly.

`GET /class-browser/query`

Runs browser queries such as implementors, senders, references, method-text search, and hierarchy-scoped variants.

Query parameters:

- `mode` — one of `implementors`, `senders`, `references`, `methodText`, `hierarchyImplementors`, `hierarchySenders`
- `selector`
- `rootClassName`
- `rootDictionary`
- `hierarchyScope` — `all`, `full`, `super`, `this`, `sub`
- `meta=1`

`GET /class-browser/file-out`

Exports dictionary/class/method source.

Query parameters:

- `mode` — `dictionary`, `class`, or `method`
- `dictionary`
- `class`
- `selector`
- `meta=1`

`POST /class-browser/inspect-target`

Resolves an inspectable OOP for helper/browser actions.

Body:

```
{
  "mode": "method",
  "dictionary": "Globals",
  "className": "Object",
  "selector": "printString",
  "meta": false
}
```

Supported modes:

- `dictionary`
- `class`
- `instances`
- `method`

Write Routes

All Class Browser write routes use `POST` JSON and return a status message in `result` when successful.

Dictionary actions:

- `POST /class-browser/add-dictionary`
- `POST /class-browser/rename-dictionary`
- `POST /class-browser/remove-dictionary`

Class actions:

- `POST /class-browser/add-class`
- `POST /class-browser/rename-class`
- `POST /class-browser/move-class`
- `POST /class-browser/remove-class`

Category actions:

- `POST /class-browser/add-category`
- `POST /class-browser/rename-category`
- `POST /class-browser/remove-category`

Variable actions:

- `POST /class-browser/add-instance-variable`
- `POST /class-browser/remove-instance-variable`
- `POST /class-browser/rename-instance-variable`
- `POST /class-browser/add-class-variable`
- `POST /class-browser/remove-class-variable`
- `POST /class-browser/rename-class-variable`
- `POST /class-browser/add-class-instance-variable`
- `POST /class-browser/remove-class-instance-variable`
- `POST /class-browser/rename-class-instance-variable`

Method actions:

- `POST /class-browser/move-method`
- `POST /class-browser/remove-method`
- `POST /class-browser/create-accessors`
- `POST /class-browser/compile`

`POST /class-browser/compile` accepts:

```
{
  "dictionary": "Globals",
  "className": "Object",
  "selector": "printString",
  "meta": false,
  "source": "displayString\n^ 'Object'",
```

```
  "sourceKind": "method"
}
```

The compile response may include `selector`, `previousSelector`, and `category` so the UI can follow method renames.

Codegen Explorer

The Codegen Explorer is an integration surface for `gemstone-py` codegen. The explorer discovers live classes, protocols/categories, methods, and source; lets users choose selectors; exports/imports selection metadata; and can preview the generated wrapper package. Code generation rules stay in `gemstone-py`.

`GET /codegen/dictionaries`

Returns visible symbol-list dictionaries for codegen discovery.

`GET /codegen/classes`

Returns classes in a dictionary.

Query parameters:

- `dictionary`

Response:

```
{
  "success": true,
  "dictionary": "Globals",
  "classes": [
    {"className": "Object", "dictionary": "Globals"}
  ]
}
```

`GET /codegen/class`

Returns instance variables, instance methods, and class-side methods for a class.

Query parameters:

- `dictionary`
- `class`

Each method includes `selector`, `category`, `argCount`, `pythonName`, and `propertyCandidate`. `propertyCandidate` is true for no-argument instance selectors that can be selected as generated fields. UI selections can override `pythonName`, `argNames`, and `returnAnnotation` before preview/export.

`GET /codegen/protocols`

Returns protocols/categories for a class or metaclass.

Query parameters:

- `dictionary`
- `class`
- `meta=1` for class side

`GET /codegen/methods`

Returns method metadata for a class or a single protocol/category.

Query parameters:

- `dictionary`
- `class`
- `protocol` optional; omit or pass `-- all --` for all protocols
- `meta=1` for class side

Each method has the same shape as `/codegen/class` method entries.

`GET /codegen/source`

Returns method source for a selector.

Query parameters:

- `dictionary`
- `class`
- `selector`
- `meta=1` for class side

`POST /codegen/preview`

Normalizes a selection, renders a Protocol draft, runs `gemstone_py.codegen.generate_package()` in a temporary directory, and returns generated file previews.

Body:

```
{
  "moduleName": "my_app.protocols",
  "async": true,
  "classes": [
    {
      "className": "Booking",
      "protocolName": "BookingProto",
      "dictionary": "Globals",
      "fields": ["status"],
      "methods": [
```

```
        {
          "selector": "markPaid:",
          "pythonName": "mark_paid",
          "argNames": ["payment"],
          "returnAnnotation": "Any"
        }
      ],
      "classMethods": []
    }
  ]
}
```

Response includes `selection`, `protocolSource`, `files`, and `warnings`.

`POST /codegen/export-selection`

Validates and returns the normalized selection metadata as JSON. This is for tools that want to persist or hand off the selected dictionaries/classes/selectors without invoking code generation. The browser uses the same endpoint to normalize imported selection JSON before restoring it into the UI.

Symbol List Browser

`GET /symbol-list/users`

Lists users from the resolved `AllUsers` collection.

`GET /symbol-list/dictionaries/<user>`

Lists symbol dictionaries for a user.

`GET /symbol-list/entries/<user>/<dictionary>`

Lists entry keys for a dictionary.

`GET /symbol-list/preview/<user>/<dictionary>/<key>`

Returns an `object_view` for the selected value.

`GET /symbol-list/debug`

Returns diagnostic text for `AllUsers` resolution.

Write Routes

- `POST /symbol-list/add-dictionary`
- `POST /symbol-list/remove-dictionary`
- `POST /symbol-list/add-entry`

- `POST /symbol-list/remove-entry`

All use JSON bodies and commit immediately on success.

Debugger

`GET /debug/threads`

Returns halted threads with summary metadata such as `sourcePreview`, `exceptionText`, and `displayText`.

`GET /debug/frames/<oop>`

Returns stack frames for the selected halted thread.

`GET /debug/frame/<oop>`

Returns source, variables, and execution-point metadata for a frame.

Query parameters:

- `index`

Frame responses include execution-point hints such as `lineNumber`, `sourceOffset`, `stepPoint`, and `ipOffset`.

`GET /debug/thread-local/<oop>`

Returns thread-local storage entries for the halted thread.

`POST /debug/proceed/<oop>`

Resumes the halted process.

`POST /debug/step-into/<oop>`

Steps into from the current frame.

`POST /debug/step-over/<oop>`

Steps over from the selected frame.

Body:

```
{"index": 0}
```

`POST /debug/trim/<oop>`

Trims the stack to the selected frame.

Body:

```
{"index": 0}
```

Transactions And Session Mode

`GET|POST /transaction/commit`

Commits the current transaction. `POST` is preferred.

`GET|POST /transaction/abort`

Aborts the current transaction. `POST` is preferred.

`GET|POST /transaction/continue`

Continues the current transaction after conflict/error handling. `POST` is preferred.

`GET /transaction/persistent-mode`

Returns the current persistent-mode flag.

`POST /transaction/persistent-mode`

Sets persistent mode.

```
{"enable": true}
```